

Distributed Computing vs Parallel Computing in detail:

Parallel Computing: In parallel computing multiple processors perform multiple tasks assigned to them simultaneously. Memory in parallel systems can either be shared or distributed. Parallel computing provides concurrency and saves time and money.

Ex: Blockchains, Smartphones, Laptop computers, Internet of Things, Artificial intelligence and machine learning, Space shuttle, Supercomputers are the technologies that uses Parallel computing technology.

Distributed Computing: In distributed computing we have multiple autonomous computers which seems to the user as single system. In distributed systems there is no shared memory and computers communicate with each other through message passing. In distributed computing a single task is divided among different computers.

Ex: Artificial Intelligence and Machine Learning, Scientific Research and High-Performance Computing, Financial Sectors, Energy and Environment sectors, Internet of Things, Blockchain and Cryptocurrencies are the areas where distributed computing is used.

Parallel Computing	Distributed Computing
Many operations are performed simultaneously	System components are located at different locations
Single computer is required	Uses multiple computers
Multiple processors perform multiple operations	Multiple computers perform multiple operations
It may have shared or distributed memory	It have only distributed memory
Processors communicate with each other through bus	Computer communicate with each other through message passing.
Improves the system performance	Improves system scalability, fault tolerance and resource sharing capabilities

Advantages of Distributed Computing:

- 1.Resource Sharing:** Uses multiple computers to share resources (CPU, storage, memory, etc.).
- 2.Scalability:** Easy to add more systems to increase performance.
- 3.Cost-Effective:** Can use inexpensive, existing machines instead of supercomputers.

Disadvantages of Distributed Computing:

- 1.Complex Communication:** Requires synchronization and coordination between nodes.
- 2.Security Issues:** More vulnerable to network attacks and data breaches.
- 3.Difficult Debugging:** Hard to trace errors across multiple machines.

Advantages of Parallel Computing:

- 1.High Speed:** Tasks are divided among multiple processors, greatly reducing execution time.
- 2.Reduced Time Complexity:** Complex algorithms execute faster when split across processors.
- 3.Better Performance:** Makes full use of CPU cores for intensive computation.

Disadvantages of Parallel Computing:

- 1.Expensive Hardware:** Requires multi-core processors or supercomputers.
- 2.Programming Complexity:** Writing parallel code and managing threads is difficult.
- 3.Synchronization Overhead:** Threads/processes need to be coordinated to avoid conflicts.

Compare JSP and Servlets with suitable examples:

Servlet: A Servlet is a Java program that runs on a web server and is used to handle requests and responses in a web application. It is part of Java EE (Jakarta EE) and extends the capabilities of a web server.

JSP: JSP is a server-side technology that allows you to embed Java code inside HTML pages. It is mainly used for presenting dynamic web content. In JSP, the `<% ... %>` tags are called scriptlets, and they allow you to write Java code inside an HTML page.

Servlet	JSP
Servlet is a Java code.	JSP is an HTML-based compilation code.
Writing code for servlet is harder than JSP as it is HTML in java.	JSP is easy to code as it is java in HTML.
Servlet is faster than JSP.	JSP is slower than Servlet because the first step in the JSP lifecycle is the translation of JSP to java code and then compile.
Servlet can accept all protocol requests.	JSP only accepts HTTP requests.
In Servlet, we can override the <code>service()</code> method.	In JSP, we cannot override its <code>service()</code> method.
In Servlet by default session management is not enabled, user have to enable it explicitly.	In JSP session management is automatically enabled.
Modification in Servlet is a time-consuming compiling task because it includes reloading, recompiling, JavaBeans and restarting the server.	JSP modification is fast, just need to click the refresh button.

SOAP (Simple Object Access Protocol): Simple Object Access Protocol (SOAP) is a network protocol for exchanging structured data between nodes. It uses XML format to transfer messages. It works on top of application layer protocols like HTTP and SMTP for notations and transmission. SOAP allows processes to communicate throughout platforms, languages, and operating system, since protocols like HTTP are already installed on all platforms.

SOAP message transmits some basic information as given below:

- Information about message structure and instructions on processing it.
- Encoding instructions for application-defined data types.
- Information about Remote Procedure Calls and their responses.

The message in XML format contains four parts:

1.Envelope: This specifies that the XML message is a SOAP message. A SOAP message is an XML document containing a header and a body, both encapsulated within the envelope. Any fault is included within the body of the message.

2.Header: This part is optional. When present, it can provide crucial information about the applications.

3.Body: This contains the actual message being transmitted. Faults are contained within the body tags.

4.Fault: This optional section contains the status of the application and any errors. It should not appear more than once in a SOAP message.

Advantages of SOAP:

- SOAP is a light weight data interchange protocol because it is based on XML.
- SOAP was designed to be OS and Platform independent.
- It is built on top of HTTP which is installed in most systems.

CODE:

Content-Type: application/soap+xml

```
<env:Envelope xmlns:env="https://www.w3.org/2003/05/soap-envelope">
  <env:Header>
    <m:GetLastTradePrice xmlns:m="Some-URI" />
  </env:Header>
  <env:Body>
    <symbol xmlns:p="Some-URI" >DIS</symbol>
  </env:Body>
</env:Envelope>
```

SOAP messaging architecture:

A SOAP message is encoded as an XML document, consisting of an <Envelope> element, which contains an optional <Header> element, and a mandatory <Body> element.

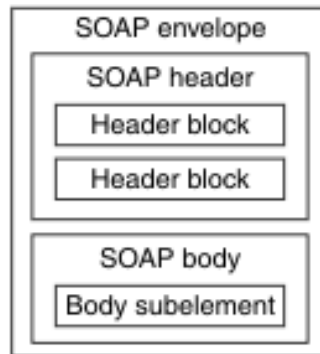
The <Fault> element, contained in <Body>, is used for reporting errors.

1.The SOAP envelope: <Envelope> is the root element in every SOAP message, and contains two child elements, an optional <Header> element, and a mandatory <Body> element.

2.The SOAP header: <Header> is an optional subelement of the SOAP envelope, and is used to pass application-related information that is to be processed by SOAP nodes along the message path.

3.The SOAP body: <Body> is a mandatory subelement of the SOAP envelope, which contains information intended for the ultimate recipient of the message.

4.The SOAP fault: <Fault> is a subelement of the SOAP body, which is used for reporting errors. XML elements in <Header> and <Body> are defined by the applications that make use of them, although the SOAP specification imposes some constraints on their structure. The following diagram shows the structure of a SOAP message.



Write short notes on JSP Directives with examples: JSP Directives provide global information to the JSP container. They control the overall structure and behavior of a JSP page but do not produce any output themselves.

Syntax: <%@ directive attribute="value" %>

Types of JSP Directives:

1.Page Directive (<%@ page %>): Provides information about the JSP page like content type, error handling, import statements, etc.

Ex:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" import="java.util.Date" %>
<html>
<body>
    <h2>Current Date and Time:</h2>
    <%
        Date d = new Date();
        out.println(d);
    %>
</body>
</html>
```

Common Attributes of <%@ page %>:

- i.language: programming language (default: Java)
- ii.contentType: MIME type of the response
- iii.import: imports Java packages/classes
- iv.errorPage: specifies a JSP page to handle errors
- v.session: indicates whether the page uses HTTP session

2.Include Directive (<%@ include %>): Used to include content of another file at translation time. Static inclusion; the included file becomes part of the JSP before compilation.

Ex:

```
<%@ include file="header.jsp" %>
<h2>Welcome to My Website</h2>
<%@ include file="footer.jsp" %>
```

3.Taglib Directive (<%@ taglib %>): Declares a custom tag library to use in the JSP page.

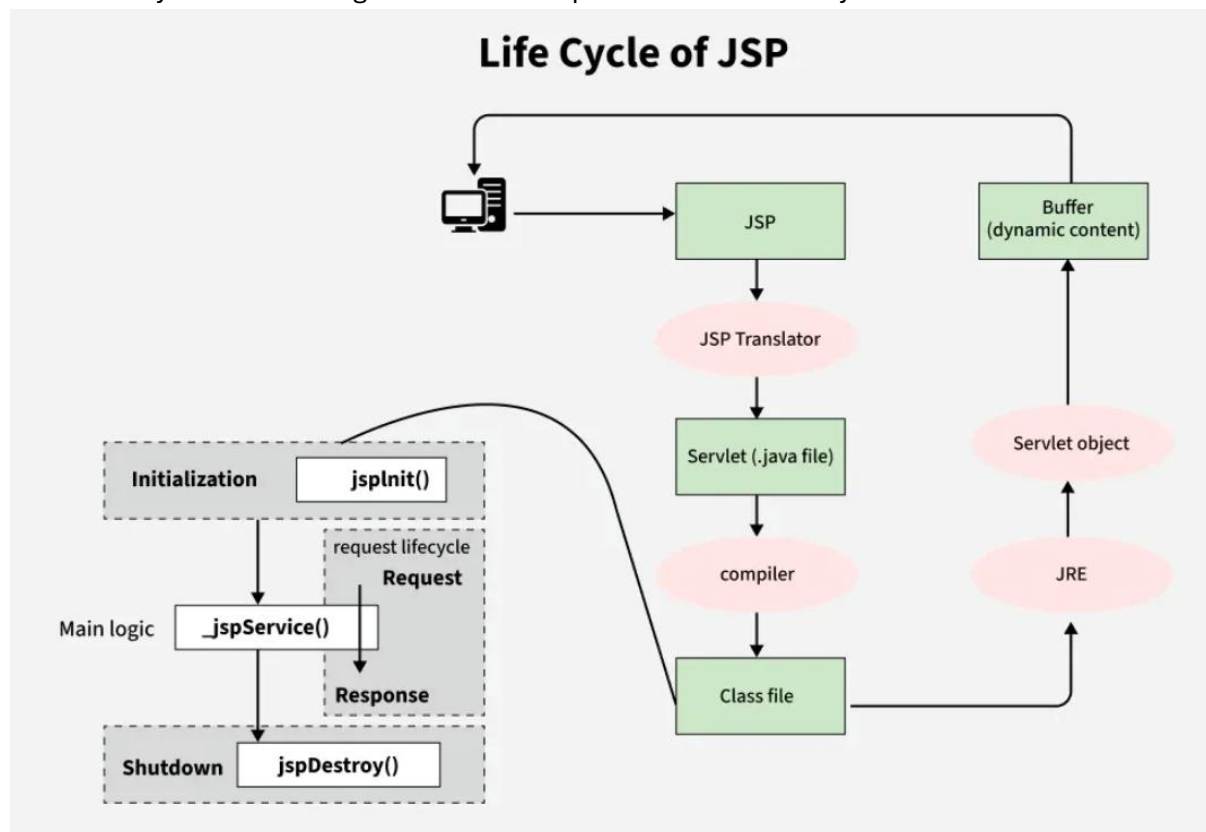
Ex:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:if test="${user != null}">
Welcome, ${user}!
</c:if>
```

Attributes:

- i. Prefix: short name to use for tags
- ii. uri: URI of the tag library

Life cycle of a JSP page: The life cycle of a JavaServer Page (JSP) consists of various phases that start from its creation, followed by its translation into a servlet, and finally managed by the servlet lifecycle. The JSP engine handles this process automatically.



1.Translation of JSP to Servlet: The JSP file is parsed and converted into a Java servlet source file (test.java). This step checks for syntax correctness.

CODE:

```
public class TestServlet extends HttpServlet {  
}
```

2.Compilation of JSP page: The generated test.java file is compiled into a class file (test.class). This step converts the servlet code into bytecode.

CODE:

```
public class TestServlet extends HttpServlet {  
}
```

3.Classloading: The container dynamically loads the compiled class.

4. Instantiation: The container creates an instance of the generated servlet class. This instance handles multiple requests unless explicitly configured otherwise.

CODE:

```
TestServlet servlet = new TestServlet();
```

5.Initialization (jspInit()): This method is called only once when the JSP is first loaded. It is used for initializing resources like database connections or configurations.

CODE:

```
public void jspInit() {  
    System.out.println("JSP Initialized.");  
}
```

6.Request Processing (_jspService()): This method is called for every request. It cannot be overridden because it is auto-generated and declared as final. It receives HttpServletRequest and HttpServletResponse objects to handle the request.

CODE:

```
public void _jspService(HttpServletRequest request, HttpServletResponse response) {  
    response.getWriter().write("<html><body>Hello, World!</body></html>");  
}
```

7.JSP Cleanup (jspDestroy()): This method is called once before removing the JSP from service. It is used for releasing resources, such as closing database connections or cleaning up open files.

CODE:

```
public void jspDestroy() {  
    System.out.println("JSP Destroyed.");  
}
```

8.This Lifecycle ensures that JSP pages are efficiently compiled, managed and cleaned up by the server container.

JSP Scripting Elements with examples: JSP Tags are special elements that simplify web page development by separating presentation from Java code. They allow developers to avoid writing Java code directly in JSP pages.

1.Comment Tag: Used for documentation in JSP. Does not appear in output.

Syntax: <%-- This is a comment --%>

2.Directive Tags: Give instructions to the JSP engine. Start with <%@ and end with %>.

Types:

i. Page Directive: Defines page attributes like language, import, session, error handling.

Ex: <%@ page language="java" import="java.util.Date" %>

ii. Include Directive: Includes another file at translation time.

Ex: `<%@ include file="header.jsp" %>`

iii. Taglib Directive: Links a custom tag library.

Ex: `<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>`

3.Scripting Tags: Allow embedding Java code in JSP.

Types:

i. Declarations `<%! ... %>`: Declare class-level variables or methods.

Ex: `<%! int count = 0; %>`

ii. Scriptlets `<% ... %>`: Write Java code statements executed for every request.

Ex: `<% count++; %>`

iii. Expressions `<%= ... %>`: Outputs the value of a Java expression directly. No semicolon required.

Ex: `<%= count %>`

`<%= Math.random() %>`